

Pathfinder — Technical Documentation

Table of Contents

1. [Architecture Overview](#)
 2. [New Machine Setup](#)
 3. [Required Package Installs](#)
 4. [Fixing npm Vulnerabilities](#)
 5. [Navigation & Screen Reference](#)
 6. [Placeholder Code Reference](#)
 7. [API Keys & Environment Variables](#)
 8. [Maps Integration](#)
 9. [iOS Swift Package Manager Setup](#)
 10. [Build & Deployment](#)
 11. [Testing](#)
 12. [Dependency Reference](#)
-

Architecture Overview

Pathfinder is a **React Native** app running on **Expo SDK 52**. It uses a single stack navigator as the root, with each feature living in its own screen component. State is local (React hooks) per screen, with cross-screen data passed via navigation params. Persistent data uses AsyncStorage.

App.tsx

```
└─ GestureHandlerRootView
  └─ NavigationContainer
    └─ Stack.Navigator (initialRoute: Login)
      └─ LoginScreen
      └─ WelcomeScreen
      └─ HomeScreen
      └─ MapScreen
      └─ FlightsScreen
      └─ HotelsScreen
      └─ ActivitiesScreen
      └─ ItineraryScreen
      └─ CalendarScreen
      └─ TravelExpense (ExpenseTracker)
      └─ BudgetScreen
      └─ CurrencyConverterScreen
      └─ BookingScreen
      └─ VisaScreen
      └─ FeedbackScreen
```

New Machine Setup

Every time you clone this repo on a new machine, run these steps in order.

Prerequisites

- Node.js 18+ → nodejs.org → download LTS
- Git → git-scm.com
- VS Code → code.visualstudio.com

Steps

```
bash
```

```
# 1. Clone
git clone https://github.com/Katechen125/Pathfinder.git
cd Pathfinder

# 2. Install all dependencies
npm install

# 3. Install Expo (required to run with QR code and Expo Go)
npm install expo

# 4. Update all packages to the correct Expo-compatible versions
npx expo install --fix

# 5. Install navigation packages (required – app will show red imports without these)
npm install @react-navigation/native @react-navigation/stack
npm install react-native-screens react-native-safe-area-context

# 6. Install config package for reading .env keys
npm install react-native-config

# 7. Install TypeScript config (fixes red tsconfig.json)
npm install --save-dev @react-native/typescript-config

# 8. Fix vulnerabilities
npm audit fix

# 9. Create your .env file (never committed to git)
# Windows:
echo. > .env
# Then open .env in VS Code and fill in your keys (see API Keys section below)
```

Change `package.json` to use Expo to start the app

Open `package.json` and find the `scripts` section. Change the `start` line from:

```
json
"start": "react-native start",
```

To:

```
json
```

```
"start": "expo start",
```

Save with `Ctrl + S`. Now `npm start` will launch Expo and show you a QR code.

Run the app

```
bash  
  
npm start
```

This shows a QR code in the terminal. Download **Expo Go** on your phone (App Store or Play Store), open it, tap **Scan QR code**, and point it at the terminal.

Windows notes:

- Use `cls` to clear the terminal instead of `clear`
- Use `findstr` instead of `grep` for searching files
- If port 8081 is busy, run `npx kill-port 8081` then try again
- The `npm warn` lines about peer dependencies are normal — they don't stop the app from running

Required Package Installs

If you see **red underlines** on any import in VS Code, it means a package is missing. Here is every package you may need to install manually and why:

Red import

Fix

<code>@react-navigation/native</code>	<code>npm install @react-navigation/native @react-navigation/stack</code>
<code>@react-navigation/stack</code>	Same as above
<code>react-native-screens</code>	<code>npm install react-native-screens react-native-safe-area-context</code>
<code>react-native-safe-area-context</code>	Same as above
<code>react-native-config</code>	<code>npm install react-native-config</code>
<code>react-native-maps</code>	<code>npm install react-native-maps</code>
<code>react-native-gesture-handler</code>	<code>npm install react-native-gesture-handler</code>
<code>react-native-reanimated</code>	<code>npm install react-native-reanimated</code>
<code>@react-native-async-storage/async-storage</code>	<code>npm install @react-native-async-storage/async-storage</code>
<code>react-native-vector-icons</code>	<code>npm install react-native-vector-icons</code>
<code>react-native-calendars</code>	<code>npm install react-native-calendars</code>
<code>react-native-chart-kit</code>	<code>npm install react-native-chart-kit react-native-svg</code>
<code>axios</code>	<code>npm install axios</code>
<code>@react-native/typescript-config</code> (tsconfig.json red)	<code>npm install --save-dev @react-native/typescript-config</code>

All of these should already be in `package.json` so `npm install` handles them automatically. If one is still red after `npm install`, run the specific command above.

Fixing npm Vulnerabilities

The safe fix (run this first, always)

```
bash
```

```
npm audit fix
```

This fixes vulnerabilities that have a clean, non-breaking update available.

What to do when vulnerabilities remain after `npm audit fix`

Some vulnerabilities cannot be auto-fixed because their fix requires a **breaking change** — meaning it would change how the app builds. These are handled with `overrides` in `package.json` instead.

The following overrides are already added to `package.json`:

```
json

"overrides": {
  "fast-xml-parser": "^4.5.3",
  "@eslint/plugin-kit": "^0.3.4"
}
```

After adding or changing overrides, run:

```
bash

npm install
```

Understanding what's actually dangerous vs what isn't

Not every vulnerability affects your users. Here is how to read the audit report:

Package	Where it lives	Risk to users
<code>fast-xml-parser</code> in <code>@react-native-community/cli</code>	Dev tool only — used to build the app, never shipped	❌ Zero
<code>@eslint/plugin-kit</code> in <code>eslint</code>	Dev tool only — runs the linter on your machine	❌ Zero
<code>axios</code> in your app code	Ships with your app	✅ Fix immediately
<code>react-native-google-places-autocomplete</code> (qs)	Ships with your app	✅ Fix immediately

Rule of thumb: if the vulnerability is inside `node_modules/@react-native-community/cli`, `node_modules/eslint`, or `node_modules/metro`, it is a dev tool and poses zero risk to your users. If it is in a package your screens directly import (axios, react-native-maps, etc.), fix it.

The `--force` loop problem

Running `npm audit fix --force` repeatedly causes an infinite loop where each fix introduces a new vulnerability in a different version of the same cli package. This is a known issue with React Native 0.78. The solution is the `overrides` approach above — do not keep running `--force`.

Checking your final vulnerability count

```
bash
npm audit
```

After all fixes and overrides, you should see only dev-tool vulnerabilities remaining (2 low, at most 3 high — all in cli/eslint packages). This is acceptable for a production React Native app.

Common Errors & Fixes

This section documents every error encountered during development and how it was resolved. Use this as a reference when something breaks.

`Cannot read properties of undefined (reading 'body')`

Cause: `express` version 5 changed how it handles request bodies and is incompatible with the current setup. **Fix:** Use `express` version 4 in `package.json`:

```
json
"express": "^4.21.0"
```

`Cannot find module '@react-native/metro-config'`

Cause: The metro config package is missing from `devDependencies`. **Fix:**

```
bash
npm install --save-dev @react-native/metro-config --legacy-peer-deps
```

Unable to resolve module react-native-worklets

Cause: `react-native-reanimated` version 4 requires `react-native-worklets` as a separate package which Expo Go does not support yet. **Fix:** Stay on version 3:

json

```
"react-native-reanimated": "~3.16.1"
```

TurboModuleRegistry.getEnforcing('PlatformConstants') could not be found

Cause: `react-native` version is incompatible with the installed Expo SDK. **Fix:** Make sure `react-native` matches the version Expo expects. Run `npx expo install --fix -- --legacy-peer-deps` to auto-correct all versions.

"main" has not been registered

Cause: `index.js` is using `AppRegistry` (React Native CLI style) instead of Expo's registration method. **Fix:** Replace the entire content of `index.js` with:

javascript

```
import { registerRootComponent } from 'expo';
import App from './App';
registerRootComponent(App);
```

Cannot read property 'getConfig' of null (react-native-config)

Cause: `react-native-config` is a native module that requires compilation into the app binary. It does not work in Expo Go. **Fix:** Replace `react-native-config` with `expo-constants` in `Source/Services/config.ts`:

typescript

```
import Constants from 'expo-constants';
const extra = Constants.expoConfig?.extra ?? {};
const AppConfig = {
  GOOGLE_MAPS_API_KEY: extra.GOOGLE_MAPS_API_KEY ?? '',
  GEOAPIFY_KEY: extra.GEOAPIFY_KEY ?? '',
} as const;
export default AppConfig;
```

And put your keys in `app.json` under the `extra` field:

json


```
"extra": {
  "GOOGLE_MAPS_API_KEY": "your_key_here",
  "GEOAPIFY_KEY": "your_key_here"
}
```

AxiosError: Request failed with status code 401 (Hotels/Activities)

Cause: The Geoapify API key was not being passed in the request params. **Fix:** Add `apiKey: GEOAPIFY_KEY` to the params in both `searchHotels` and `searchActivities` in `API.ts`:

```
typescript

params: {
  categories: 'accommodation',
  filter: `circle:${lng},${lat},5000`,
  limit: 20,
  apiKey: GEOAPIFY_KEY,
}
```

AxiosError: Request failed with status code 400 (Hotels/Activities)

Cause: The API URL had hardcoded example params AND a `params` object — they conflicted. **Fix:** Use only the base URL with a clean `params` object, no hardcoded query strings in the URL.

Geocoding error: No results found

Cause: The search term passed to `geocodeLocation` is empty or Google Maps returns no results for it. The app handles this gracefully by falling back to mock coordinates. **Fix:** This is non-fatal — the app continues running with mock data. To reduce occurrences, make sure the user searches a valid city or country name.

Past searches not clickable (HomeScreen)

Cause: The past search cards only had a delete button — the card itself had no `onPress` handler. **Fix:** Wrap the entire search card in a `TouchableOpacity` that sets `searchQuery` and closes the dropdown when tapped.

Project is incompatible with this version of Expo Go

Cause: Expo Go on iPhone only supports the latest SDK. If your project uses an older SDK, it won't open. **Fix:** Upgrade your project to match the SDK version Expo Go requires:

```
bash
```

```
npm install expo@54 --legacy-peer-deps
npx expo install --fix -- --legacy-peer-deps
```

Then set `"sdkVersion": "54.0.0"` in `app.json`.

Running the App with Expo Tunnel

What `--tunnel` does

Normally Expo tries to connect your phone to your laptop over your local WiFi network. This often fails on Windows because of firewall rules that block the connection — which is why you get "Could not connect to server."

`--tunnel` solves this by routing the connection through Expo's servers on the internet instead of your local network. Your phone connects to Expo's cloud, which then connects to your laptop. This works even when WiFi direct connection fails.

When to use it

Always use `--tunnel` on Windows when testing on a physical iPhone or Android device:

```
bash
npx expo start --tunnel
```

Why `--clear` helps

Metro (the JavaScript bundler) caches files to speed up reloads. Sometimes the cache gets stale and causes errors. Adding `--clear` wipes the cache on startup:

```
bash
npx expo start --tunnel --clear
```

Use `--clear` whenever you see a strange error that wasn't there before, or after changing `package.json`.

ngrok requirement

Tunnel mode requires `@expo/ngrok` to be installed globally. If you get a tunnel error:

```
bash
```

```
npm install -g @expo/ngrok@^4.0.0
```

react-native-reanimated Version Warning

Why this warning keeps showing

`react-native-reanimated` version 4 introduced a breaking change — it split its animation engine into a separate package called `react-native-worklets`. Expo Go does not include the native `react-native-worklets` binary yet, so using reanimated v4 causes the app to crash with:

```
Unable to resolve module react-native-worklets
```

The fix — stay on version 3

This project uses `react-native-reanimated` version `~3.16.1` which has worklets built in and is fully compatible with Expo Go and SDK 54. This is intentional.

```
json

"react-native-reanimated": "~3.16.1"
```

Why the warning still appears

Even with v3 installed, Expo's compatibility checker may still show a yellow warning suggesting you upgrade to v4. **Ignore this warning** — upgrading to v4 will break the app in Expo Go. The warning is cosmetic and does not affect how the app runs.

When it is safe to upgrade to v4

Only upgrade to `react-native-reanimated` v4 when:

- Expo Go officially supports `react-native-worklets` (check expo.dev/changelog)
- Or you switch from Expo Go to a custom development build using EAS Build

Navigation & Screen Reference

Navigation uses `@react-navigation/native` with `createStackNavigator`.

Route Type Map (Source/Services/types.ts)

```
typescript

export type RootStackParamList = {
  Login: undefined;
  Welcome: undefined;
  Home: undefined;
  Map: undefined;
  Flights: undefined;
  Itinerary: undefined;
  Hotels: undefined;
  Activities: undefined;
  Visa: undefined;
  Calendar: undefined;
  ExpenseTracker: undefined;
  CurrencyConverter: undefined;
  Booking: undefined;
  Budget: undefined;
  Feedback: undefined;
};
```

Navigating Between Screens

```
typescript

import { useNavigation } from '@react-navigation/native';
import { StackNavigationProp } from '@react-navigation/stack';
import { RootStackParamList } from '../Services/types';

type Nav = StackNavigationProp<RootStackParamList>;
const navigation = useNavigation<Nav>();
navigation.navigate('Map');
```

Screen Summary

Screen	File	Status
Login	LoginScreen.tsx	⚠️ Mock auth
Welcome	WelcomeScreen.tsx	✅ Display only
Home	HomeScreen.tsx	✅ Navigation hub

Screen	File	Status
Map	MapScreen.tsx	✓ Live (needs key)
Flights	FlightScreen.tsx	✓ Live (needs SerpAPI key)
Hotels	HotelsScreen.tsx	✓ Live (needs SerpAPI key)
Activities	ActivitiesScreen.tsx	✓ Live (needs SerpAPI key)
Itinerary	ItineraryScreen.tsx	✓ AsyncStorage
Calendar	CalendarScreen.tsx	✓ Device calendar
Expense Tracker	TravelExpense.tsx	✓ Local storage
Budget	BudgetScreen.tsx	✓ Local storage
Currency Converter	CurrencyConverterScreen.tsx	⚠ Free/static rates
Booking	BookingScreen.tsx	⚠ Mock data
Visa	Visa.tsx	⚠ Static dataset
Feedback	FeedbackScreen.tsx	⚠ No backend

Placeholder Code Reference

These are the screens that contain dummy, static, or incomplete logic.
The markers below show exactly what pattern to look for in each file.

⚠ LoginScreen.tsx — No Real Authentication

What it does now: Stores a username in local state or AsyncStorage. No password hashing, no token, no server call.

What to look for in the file:

```
typescript
```

```
// Signs of mock auth – look for any of these:
const [username, setUsername] = useState('');
AsyncStorage.setItem('user', username); // just saving a string
navigation.navigate('Welcome'); // navigates without any validation
// OR hardcoded credentials:
if (password === '1234') { ... }
```

What real auth looks like:

typescript

```
// Replace with a real auth call, e.g. Firebase Auth:
import auth from '@react-native-firebase/auth';
await auth().signInWithEmailAndPassword(email, password);
// Or your own backend:
const response = await axios.post('https://yourapi.com/login', { email, password });
const { token } = response.data;
await AsyncStorage.setItem('authToken', token);
```

⚠ BookingScreen.tsx — Mock Booking Data

What it does now: Almost certainly displays a hardcoded or AsyncStorage list of items selected from Flights/Hotels screens. No real booking API.

What to look for:

typescript

```
// Mock data pattern:
const bookings = [
  { id: '1', type: 'flight', from: 'JFK', to: 'LAX', price: '$299' },
  { id: '2', type: 'hotel', name: 'Grand Hotel', nights: 3 },
];
// Or reading from AsyncStorage but no payment/confirmation flow
```

What to implement:

- A real booking API (Amadeus, Booking.com affiliate API, etc.)
- Payment integration (Stripe, etc.)
- Confirmation number / reference storage

⚠ CurrencyConverterScreen.tsx — Static or Free Rate Source

What it does now: Likely uses a free open exchange rates endpoint or has hardcoded rates baked into the file.

What to look for:

```
typescript

// Hardcoded rates:
const rates = { USD: 1, EUR: 0.92, GBP: 0.79, JPY: 149.5 };

// OR a free API with no key:
fetch('https://open.er-api.com/v6/latest/USD')
// OR using a key that should be in .env:
fetch(`https://api.exchangerate-api.com/v4/latest/USD?apikey=HARDCODED_KEY_HERE`)
```

What to do:

- If there's a hardcoded key → move it to `.env` as `EXCHANGE_RATE_API_KEY`
- If it's completely hardcoded rates → add a real free API call (Open Exchange Rates free tier is fine for this)
- Update `.env.example` with the new key

⚠ Visa.tsx — Static Dataset

What it does now: Almost certainly a hardcoded object or array mapping country pairs to visa requirements.

What to look for:

```
typescript

// Static data pattern:
const visaData: Record<string, string> = {
  'US-FR': 'No visa required for stays under 90 days',
  'US-CN': 'Visa required – apply at Chinese embassy',
  // ...
};
```

What to implement:

- A live visa API (Sherpa API, Visalist API, or Amadeus Travel Restrictions)
 - Or: keep static data but add a "last updated" date and document it clearly
-

⚠ FeedbackScreen.tsx — No Backend Submission

What it does now: Probably saves feedback to AsyncStorage or just `console.log`s it. No server receives the feedback.

What to look for:

```
typescript

// Local-only pattern:
console.log('Feedback submitted:', { rating, comment });
// OR
await AsyncStorage.setItem('lastFeedback', JSON.stringify({ rating, comment }));
navigation.goBack();
```

What to implement:

- A backend endpoint (your own API, Firebase Firestore, Supabase, etc.)
- Or a simple service like Formspree or Airtable for lightweight feedback collection:

```
typescript

await axios.post('https://formspree.io/f/YOUR_FORM_ID', {
  rating,
  comment,
  timestamp: new Date().toISOString(),
});
```

API Keys & Environment Variables

How keys are loaded

Keys are stored in a `.env` file (never committed) and read via `react-native-config`:

```
typescript
```



```
import Config from 'react-native-config';
const key = Config.GOOGLE_MAPS_API_KEY;
```

Required keys

Variable	Used In	Get It From
GOOGLE_MAPS_API_KEY	MapScreen, app.json	Google Cloud Console
SERPAPI_KEY	FlightScreen, HotelsScreen, ActivitiesScreen	serpapi.com/dashboard

Optional keys (if you implement real versions of placeholder screens)

Variable	Used In	Get It From
EXCHANGE_RATE_API_KEY	CurrencyConverterScreen	exchangerate-api.com

Restricting your Google Maps key

In [Google Cloud Console](#) → [APIs & Services](#) → [Credentials](#):

1. Click your key → **Edit**
2. **Application restrictions** → iOS apps + Android apps (add your bundle ID / package name)
3. **API restrictions** → Restrict to: Maps SDK for iOS, Maps SDK for Android, Places API
4. Save

Maps Integration

MapScreen pattern

typescript

```

import MapView, { Marker } from 'react-native-maps';
import { GooglePlacesAutocomplete } from 'react-native-google-places-autocomplete';
import Config from 'react-native-config';

// Place search
<GooglePlacesAutocomplete
  placeholder="Search places..."
  onPress={(data, details) => {
    const { lat, lng } = details.geometry.location;
    // move map to this location
  }}
  query={{
    key: Config.GOOGLE_MAPS_API_KEY, // ← from .env, NOT hardcoded
    language: 'en',
  }}
  fetchDetails={true}
/>

// Map view
<MapView
  style={{ flex: 1 }}
  initialRegion={{
    latitude: 25.7617,
    longitude: -80.1918,
    latitudeDelta: 0.0922,
    longitudeDelta: 0.0421,
  }}
/>

```

Expo Migration Notes

What changed from the original React Native CLI project

Before	After
React 19.0.0	React 18.3.1 (Expo 52 requires this)
react-native 0.78.1	react-native 0.76.5 (Expo 52 compatible)
<code>@react-native-community/cli</code> bootstrap	Expo SDK 52

Before

After

`metro.config.js` defaults

`babel-preset-expo`

Manual CocoaPods

Expo manages native layer; SPM for Google Maps

`jest.config.js` (custom)

`jest-expo` preset

No `GestureHandlerRootView`

Required wrapper added to `App.tsx`

Minimal `app.json`

Full Expo plugin config

No `.env` protection

`.env` + `react-native-config`

iOS Swift Package Manager Setup

Remove CocoaPods

```
bash
cd ios
pod deintegrate
cd ..
```

Delete: `ios/Podfile`, `ios/Podfile.lock`, `ios/Pods/`

Add Google Maps via SPM in Xcode

1. Open `ios/Pathfinder.xcodeproj` in Xcode
2. **File** → **Add Package Dependencies...**
3. Add `https://github.com/googlemaps/ios-maps-sdk` (min version 9.0.0)
4. Add `https://github.com/googlemaps/ios-places-sdk` (min version 9.0.0)
5. Select your app target for both

Initialize in AppDelegate

```
swift
```

```
// ios/Pathfinder/AppDelegate.swift
import GoogleMaps

// In application(_:didFinishLaunchingWithOptions:)
GMServices.provideAPIKey("YOUR_RESTRICTED_IOS_KEY")
```

The key here goes directly in the native file (not via react-native-config). Make sure it is an **iOS-restricted** key from Google Cloud Console.

Build & Deployment

Development

```
bash

npm start          # Expo dev server
npm run ios
npm run android
```

Production with EAS

```
bash

npm install -g eas-cli
eas login
eas build:configure # first time only

# Add secrets (never stored in repo)
eas secret:create --scope project --name GOOGLE_MAPS_API_KEY --value "AIza..."
eas secret:create --scope project --name SERPAPI_KEY --value "..."

# Build
eas build --platform ios
eas build --platform android

# OTA updates
eas update --branch production --message "Description of update"
```

Testing

```
bash

npm test           # Run all Jest tests
npm test -- --coverage  # With coverage report
npm run type-check   # TypeScript check only
```

Adding a test

```
typescript

// __tests__/HomeScreen.test.tsx
import React from 'react';
import { render } from '@testing-library/react-native';
import HomeScreen from '../Source/Screens/HomeScreen';

describe('HomeScreen', () => {
  it('renders without crashing', () => {
    const { getByTestId } = render(<HomeScreen />);
    expect(getByTestId('home-screen')).toBeTruthy();
  });
});
```

Dependency Reference

Package	Purpose	Security Note
<code>expo</code> ~52	SDK and toolchain	
<code>react-native-maps</code>	Map view	Needs Maps API key
<code>react-native-google-places-autocomplete</code>	Place search	Needs Places API key
<code>@react-navigation/native</code> + <code>stack</code>	Navigation	
<code>@react-native-async-storage/async-storage</code>	Local persistence	
<code>axios</code> ^1.9.0	HTTP requests	v1.9.0 fixes <code>__proto__</code> DoS

Package	Purpose	Security Note
<code>react-native-reanimated</code>	Animations	Needs Babel plugin
<code>react-native-gesture-handler</code>	Gestures	Needs <code>GestureHandlerRootView</code>
<code>react-native-config</code>	.env injection	
<code>serpapi</code>	Search API (flights/hotels)	Needs SerpAPI key
<code>tar</code> ^7.4.3	Archive handling	Path traversal fix
<code>minimatch</code> ^9.0.5	Glob patterns	ReDoS fix
<code>node-forge</code> ^1.3.1	Crypto	ASN.1 fix
<code>undici</code> ^6.21.1	HTTP client	WebSocket fixes
<code>fast-xml-parser</code> ^4.5.0	XML parsing	Entity expansion fix
<code>qs</code> ^6.14.0	Query strings	DoS fix